RUB

RUHR-UNIVERSITÄT BOCHUM

MEMORY

Computer Architecture

Agenda

- The Memory Hierarchy
- Memory Read/Write
- Volatile Memory
- Non-volatile Memory
- Error correction and detection

Why Are There Different Kinds of Memory?

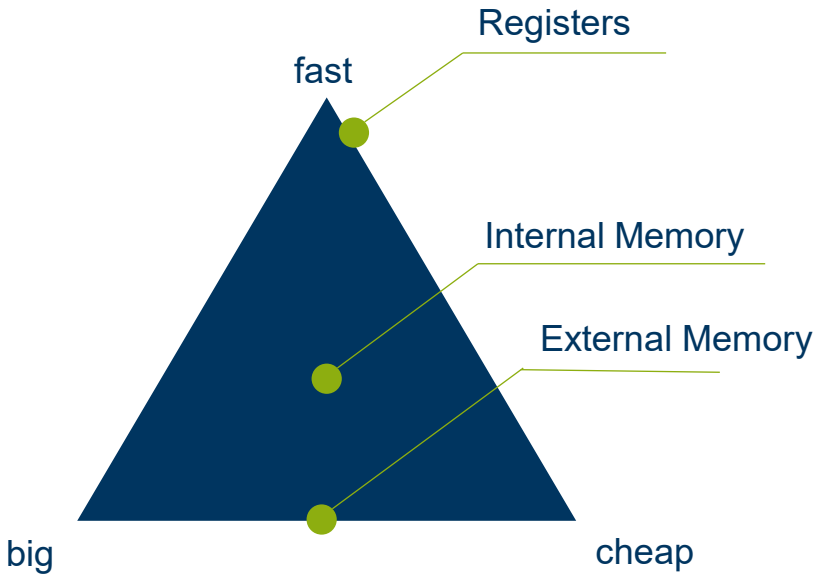
Different Memory has different attributes:

- Cheap/expensive
- Short/long access times
- small/big size

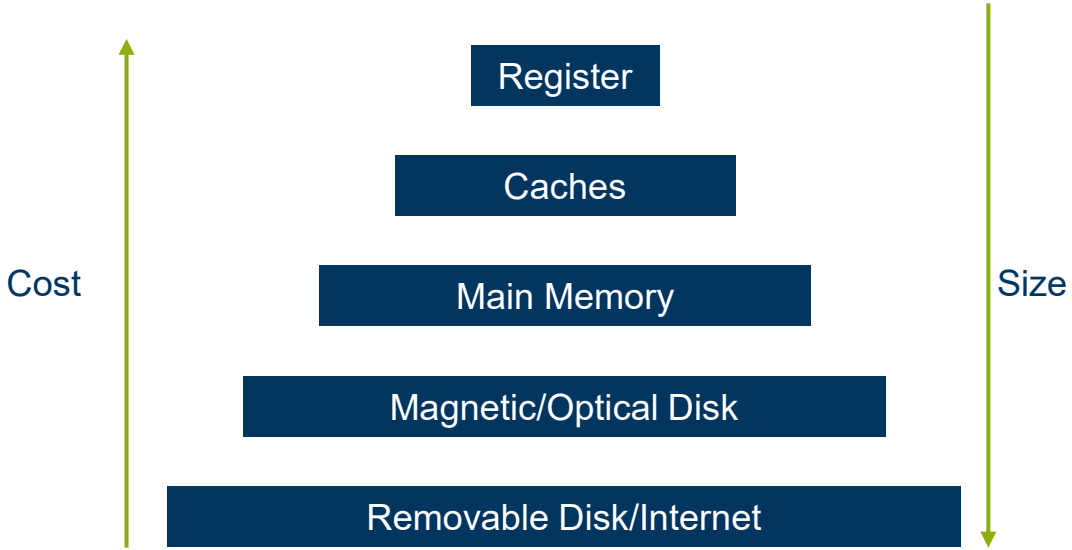
Requirement:

Need lots of memory and it has to be fast and cheap

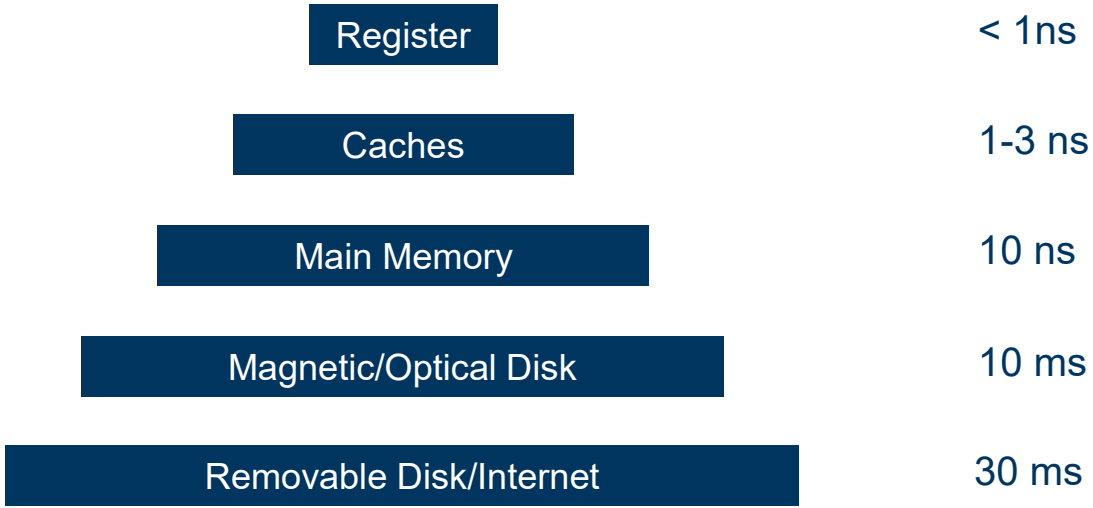
➡ No single type of memory has all three



A Compromise: Memory Hierarchy



Access Speeds



Sequential Access

Data can be stored as blocks that have to be accessed sequentially

Skips are possible but the read order is always the same

Example: Magnetic tape

Random Access

Any memory address can be accessed at any time

Examples - next

Random Access

Any memory address can be accessed at any time

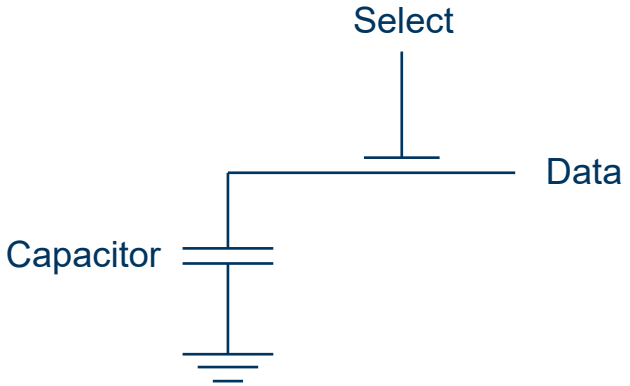
Volatile Memory

Dynamic Random Access Memory (DRAM)

Data is read from the capacitor's charge

Capacitor loses its charge over time

➡ Needs to be refreshed every 10 to 100 ms



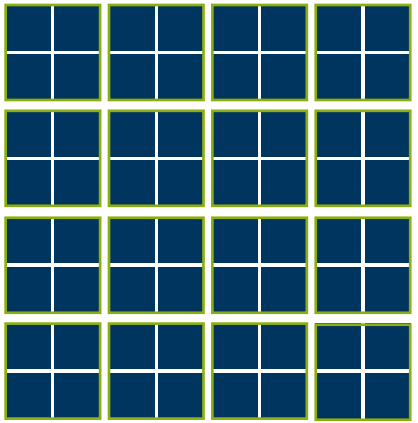
DRAM cell to store 1 bit

Cell Matrices

Each DRAM chip consists of n "supercells"

Each "supercell" consists of m cells and has the address (i, j) with i indicating the row and j the column

➡ $n \times m$ cells per chip



Here, each of the 16 supercells consists of 4 cells

And now it's your turn…

Consider a DRAM chip where the "supercells" are positioned in an 8 by 8 grid. The whole chip consists of 1024 cells.

How many cells are there per "supercell"?

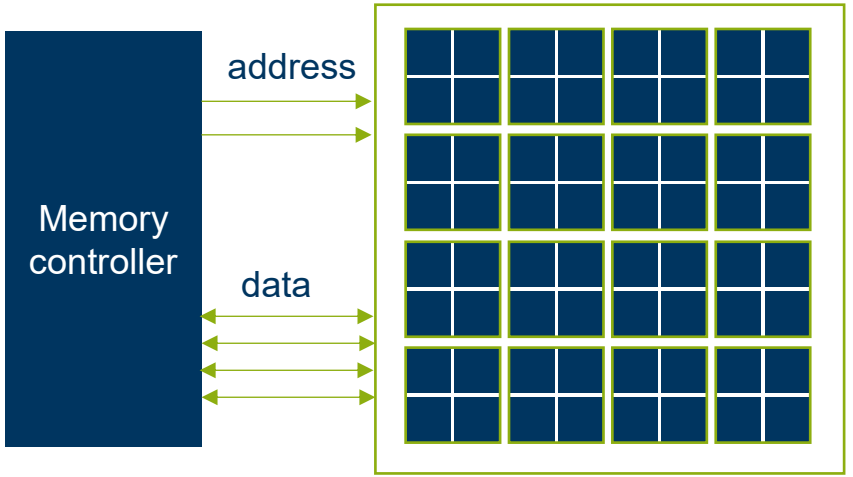
Memory Controller

Chip is connected to memory controller via pins

Each pin carries a 1 bit signal

Here:

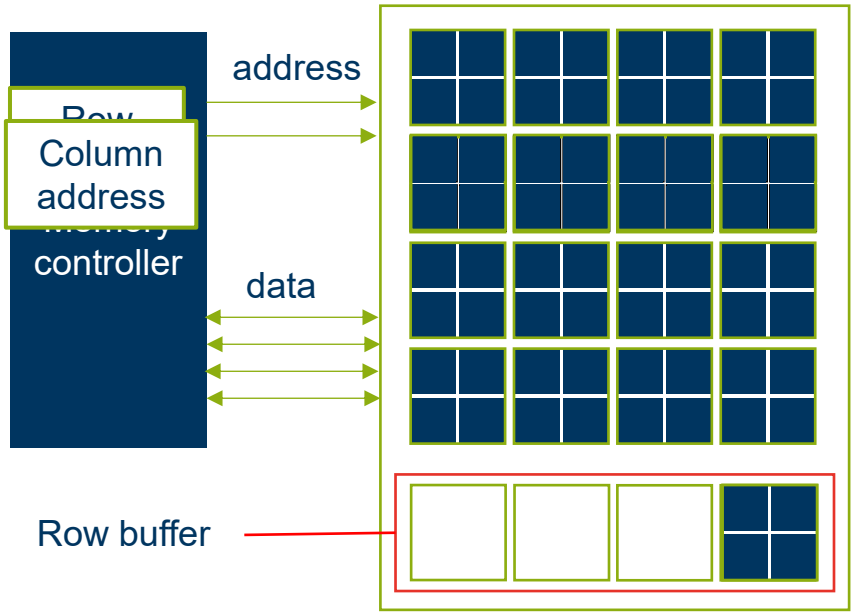
- 2 pins for address
- 4 pins for data



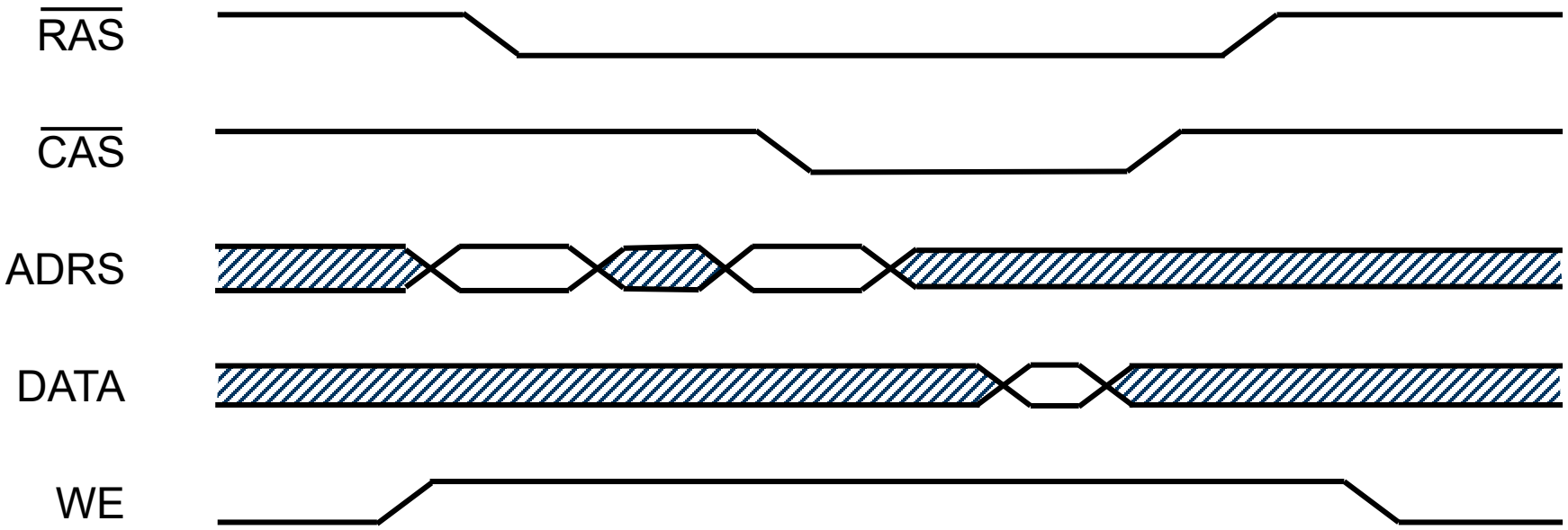
DRAM Operation

Access:

1. Memory controller sends row address
2. Row is loaded into row buffer
3. Memory controller sends column address
4. Chip sends content of supercell at that position from row buffer to memory controller
5. Row buffer written back to row



DRAM Access Timing



Synchronous DRAM (SDRAM)

- Use a common clock for memory controller and DRAM

- Memory controller communicates with DRAM on rising clock edge
  - $\overline{\text{RAS}}$, $\overline{\text{CAS}}$, and WE become a command
- No need to assert the signals for the duration of the operation
- Achieve parallelism by using multiple banks
  - Identified by additional “address” lines
- Double Data Rate SDRAM (DDR)
  - Sends data on rising and on falling clock edge

SDRAM Memory Timing

- SDRAM timing specified as 3 (sometimes 4) numbers separated by dashes
- Format: CL-T_{RCD}-T_{RP}
  - CL: CAS Latency
  - T_{RCD}: Row to column delay
  - T_{RP}: Row precharge time
- Typical memory latency:

$$2 \cdot CL \cdot \frac{1000}{Frequency}$$

DESCRIPTION

Kingston FURY KF432C16BB1K2/32 is a kit of two 2G x 64-bit (16GB) DDR4-3200 CL16 SDRAM (Synchronous DRAM) 2Rx8, memory module, based on sixteen 1G x 8-bit FBGA components per module. Each module kit supports Intel® Extreme Memory Profiles (Intel® XMP) 2.0. Total kit capacity is 32GB. Each module has been tested to run at DDR4-3200 at a low latency timing of 16-18-18 at 1.35V. The SPDs are programmed to JEDEC standard latency DDR4-2400 timing of 17-17-17 at 1.2V. Each 288-pin DIMM uses gold contact fingers. The JEDEC standard electrical and mechanical specifications are as follows:

FACTORY TIMING PARAMETERS

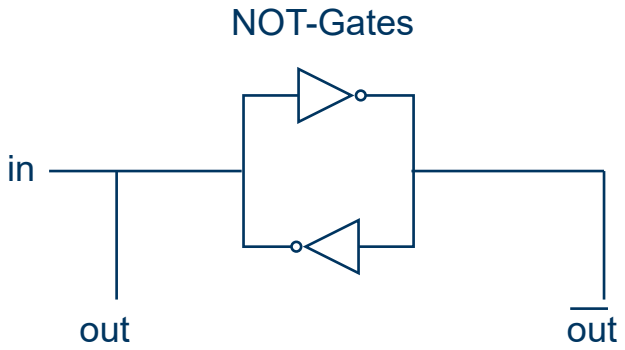
- Default (JEDEC): DDR4-2400 CL17-17-17 @ 1.2V
- XMP Profile #1: DDR4-3200 CL16-18-18 @ 1.35V
- XMP Profile #2: DDR4-3000 CL15-17-17 @ 1.35V

Static Random Access Memory (SRAM)

Data is read from the state of the inner transistors

Cells are bit-stable: once a state is entered it stays in that state until an outside signal to change occurs

No refreshes are needed



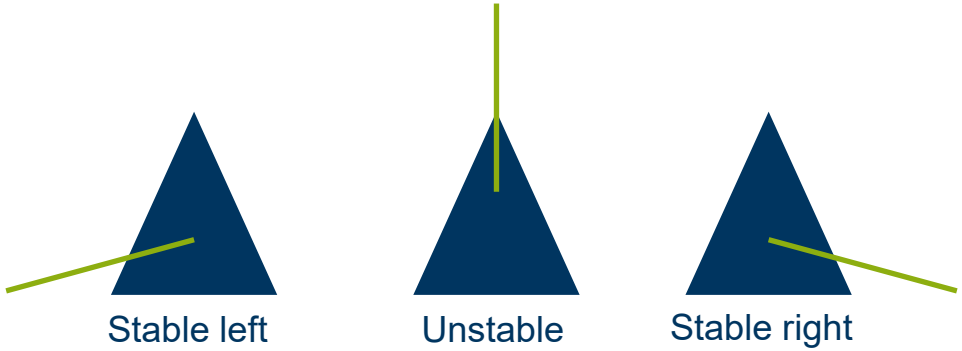
Bit-stable: Pendulum Model

Pendulum is stable when it is turned either to the left or right

➡ SRAM cell holds the charge it is set to

The unstable state can occur, but the smallest force would make it fall into a stable state

➡ Electrical interference can disturb the cell but it will go back in a defined state



SRAM versus DRAM

SRAM:

- expensive
- one cell requires many transistors
- no refresh needed
- faster
- insensitive to disturbances

Used for cache memory

DRAM:

- cheap
- one cell requires few transistors
- refresh needed
- slower
- sensitive to disturbances

Used for main memory

Why Volatile?

Both SRAM and DRAM lose their charge if electricity is gone

If we want to keep the data, we either have to never turn our PC off or try something else…

Non-Volatile Memory

Non-Volatile Random Access Memory (NVRAM)

A simple approach:

Combine a previous memory technology with a puffer battery

SRAM needs no refreshes and therefore less energy

This type of NVRAM can hold the necessary charge for keeping data for up to 10 years

NVRAM Limitations

Eventually NVRAM cells will lose their charges, and the data is lost

However, there is another approach:

Design Memory in such a way that the data that is brought onto the chip can only be removed if it is actively deleted

EPROM and EEPROM

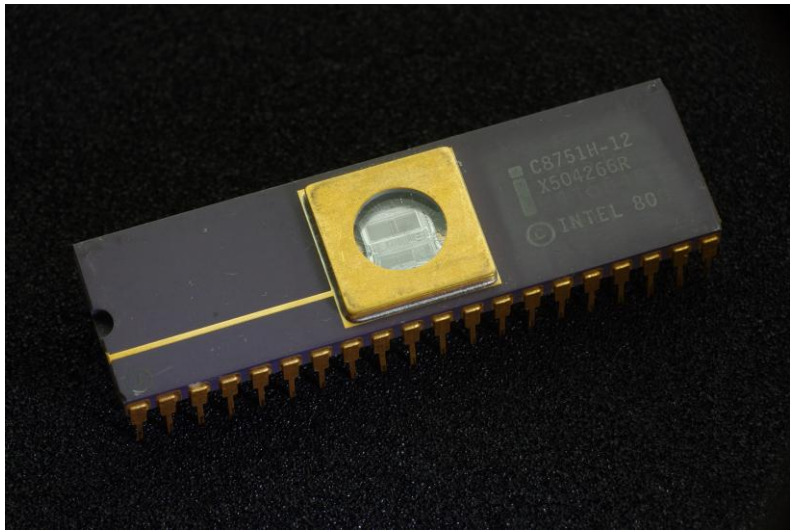
Erasable Programmable Read Only Memory (EPROM)

Transistors have a floating gate that can only be charged with very high voltages

Data on the chip can be deleted using UV-Light 100-200 times

Cells cannot be individually erased

One erase takes 10-30 minutes



yellowcloud/ CC BY 2.0 via Wikimedia Commons

Electrically Erasable Programmable Read Only Memory

Memory can be erased with electric current

The current necessary to activate the floating gate can be generated by the chip

Individual bits or blocks can be erased

Data can be erased 10 000 to 1 000 000 times

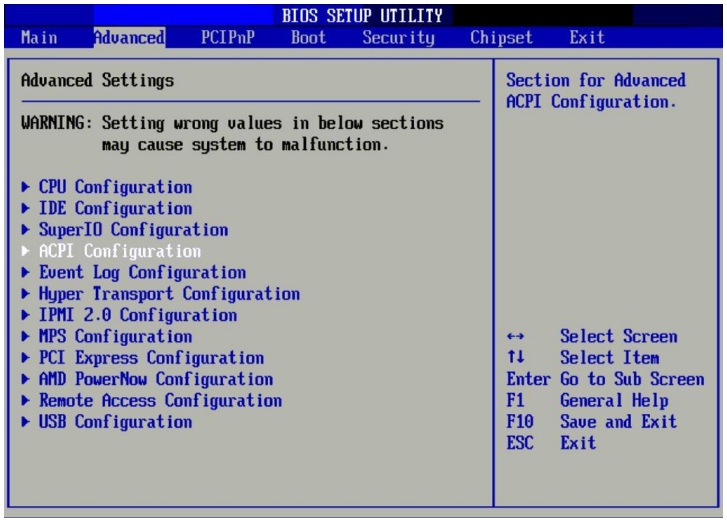
A Use For NV-Memory: BIOS/UEFI

Basic Input Output System / Unified Extensible Firmware Interface

Firmware and configuration data

Initializes the hardware components and starts the operating system

Has to be saved even if power is turned off for a long time



Error Correction and Detection

Motivation

It cannot be warranted that a complete stream of data will be transmitted correctly all the time

E.g. CD scratches, transmission noise…

Errors need to be detected

And what then?

Ask the sender to resend the data?

OR: Correct the errors ourselves

Detecting Errors

Naive solution: Send each bit twice

0101101110001  00110011110011111100000011

If the corresponding bits are different an error occurred

Disadvantage: Very high level of redundancy

Sending everything twice is not efficient at all

Detecting Errors: Parity Check

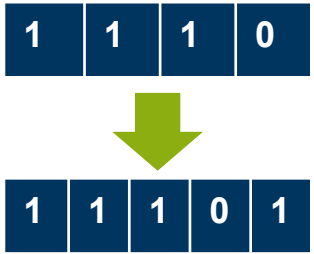
A more efficient approach is needed:

Count the number of bits set to 1 in our data stream

Append one "parity bit" and…

… set it to one if the number of ones in the stream is odd

… set it to zero if the number of ones in the stream is even



-> If the data is received and the number of ones is odd, an error has occurred

Detecting Errors: Parity Check

For this approach, the stream is usually divided into words of fixed length (e.g. bytes) and a parity bit is added to each word

Here we only append one extra bit for each word: more efficient

If an error occurrs, the sender only needs to resend the word that contains the error

Detecting Errors: Parity Check

What if two errors happen?

→ Number of ones is still even (Error is not detected)

Data may or may not make sense

What if three errors happen?

→ Number of ones is still odd (Error is detected)

This mechanism cannot rule out that no error occurred or that only one error occurred

Detecting More than One Error

- Let W^n be the set of binary words of length n
- The Hamming weight of a word is the number of 1-bits in it
  - Example: HW(00101101)=4
- The Hamming distance between two words $w_1, w_2 \in W^n$ is the number of bits where the words differ
  - Example: HD(001011, 101001)=2
- Conversion:
  - $\mathrm{HW}(w) = \mathrm{HD}(w, 0^n)$
  - $\mathrm{HD}(w_1, w_2) = \mathrm{HW}(w_1 \oplus w_2)$

Detecting More than One Error

- A code C is an injective function from W^m into W^n
  - Example: repeating bits is a code from W^m into W^{2m}
  - Example: A parity bit is a code from W^m into W^{m+1}

- The distance of a code C is the minimum Hamming distance between code words:

$$\mathrm{d_{min}} = \min\left(\mathrm{HD}\left(C(w_1), C(w_2)\right)\right)$$

- To detect up to k errors, we need a code with distance $k + 1$
  - A code with distance 3 is required to detect 2 errors

Code with Distance 3

- Repeat every bit three times

  010  000111000

- Can detect any two errors
- Can alternatively correct any single error
  - If we receive 010111000 we can guess where the error is
- More generally, a code with distance $2k + 1$ can correct up to k errors
  - Find the closest code word

Minimum Overhead for Distance 3 Codes

- Can detect every error of one bit
- There are at least n possible errors that we can correct to a given code word
- $2^m \leq {}^{2^n}/_{(n+1)}$
- $\log 2^m \leq \log\left({}^{2^n}/_{(n+1)}\right)$
- $m \leq n - \log(n+1)$
- Can we do that?

Correcting Errors: Hamming-Code

Multiple parity bits are used: all bits are shifted towards the MSB to make space in the positions that are powers of two

Position:	11	10	9	8	7	6	5	4	3	2	1
Bit:	0	1	0	1	0	1	0	1	0	1	0

Position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Bit:	0	1	0	1	0	1	0	_	1	0	1	_	0	_	_

Encoding: Computing Parity Bits

For each codebit n:
If the codebit is set to 0 fill all parity bits needed to make up the position of n with 0 otherwise with 1

E.g. bit 7 is set to 1, 7 = 4+2+1
therefore, in this row ones are put into the columns of parity bits 4, 2 and 1

Finally, add the numbers in each parity bit column, discarding any overflow

Parity bits →	8	4	2	1	Code ↓
3			0	0	0
5		1		1	1
6		0	0		0
7		1	1	1	1
9	0			0	0
10	1		1		1
11	0		0	0	0
12	1	1			1
13	0	0		0	0
14	1	1	1		1
15	0	0	0	0	0
	1	0	1	0	

Endcoding: Computing Parity Bits

Position:	11	10	9	8	7	6	5	4	3	2	1
Bit:	0	1	0	1	0	1	0	1	0	1	0

Position:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Bit:	0	1	0	1	0	1	0	1	1	0	1	0	0	1	0

Error-Checking

We recieve: 010000111110101

Did an error occur?

Recompute the parity bits and compare them with the ones that were sent

Computed: 0000

Sent: 1001

The two sets of bits differ in positions 8 and 1

most likely an error occured in position 8+1 = 9

Parity bits →	8	4	2	1	Code ↓
3			1	1	1
5		1		1	1
6		1	1		1
7		1	1	1	1
9	1			1	1
10	0		0		0
11	0		0	0	0
12	0	0			0
13	0	0		0	0
14	1	1	1		1
15	0	0	0	0	0
	0	0	0	0	

Hamming Code: Limitations

The Hamming Code can detect up to two errors and is able to correct one, if only one error happened

As with the parity check method, it cannot be ruled out that multiple errors occured or that there was no error at all

Summary

- Different types of memory is used for different usecases
- On some types every bit can be accessed at any time while other types require the data to be accessed in a specified order
- DRAM is volatile memory where all cells have to be refreshed over time
- SRAM is volatile memory that holds its charge
- Memory can be non-volatile by using physical effects, batteries or by permanently setting charges that can be only undone by erasing them
- Components like the BIOS or the UEFI have to be saved in Non-Volatile memory
- Errors in data transmissions can be detected by various methods, you learned how parity check for error correction and the hamming code for error correction and detection worked